Neste EA2, vamos simular um sistema de defesa aérea de mísseis, conforme descrição abaixo. Temos uma cidade, que pode ter defesas, e pode sofrer dano de ataques de mísseis inimigos.

Cada execução do programa representa uma única cidade retangular com dois vértices opostos: a origem (0,0) do plano cartesiano e o vértice (X,Y). Isto é, X é a "largura" da cidade e Y é a "altura" da mesma. Cada cidade pode possuir métodos de defesa aérea e pode registrar mísseis de ataque que entraram em seu radar.
Defesa aérea. A defesa cobre uma área circular da cidade. Serão fornecidos as seguintes informações de uma defesa aérea:

- Identificador único. Exemplos: 1, ABC123, etc (máx 100 caracteres)
- Coordenada X e Y em que a defesa é posicionada na cidade (centro da região). Considere que a defesa cadastrada sempre estará dentro da cidade (isto é, não é necessário verificar a posição).
- Tipo da defesa aérea: Sempre será "C", indicando circunferência
- Tamanho de proteção. Esse valor determina a área de proteção da defesa aérea. Se a defesa for do tipo C, o tamanho representa o raio de uma circunferência.
- Poder de neutralização do míssil de defesa
- Quantidade de mísseis de defesa

Míssil de ataque: é um projétil que tem como objetivo atingir um ponto (X, Y) do mundo. Serão fornecidas as seguintes informações:

- Identificador único. Exemplos: 1, ABC123, etc (máx 100 caracteres)
- Coordenada X e Y de ataque. Neste caso, o míssil pode ter como alvo um ponto fora da cidade.
- Poder de ataque do míssil

<u>Regras de funcionamento</u>

- Sua main deve ter o seguinte comportamento:
    1. Recebe as informações da cidade (X e Y)
    2. Lê da entrada, se houver, informações dos mísseis de ataque detectados e das defesas, em qualquer ordem e em qualquer número de cadastros.
    3. Realiza o processamento dos mísseis cadastrados, se houverem.
    4. Imprime o relatório conforme especificação;
- O processamento dos mísseis consiste em verificar cada ataque de míssil (isto é, verificar se o alvo do míssil está dentro da cidade), e em caso positivo 3 situações podem ocorrer:
    1. O míssil não perde nenhum poder de destruição e a cidade leva um dano igual ao poder do míssil. Isso ocorrerá nas seguintes situações:

1.1- O míssil atinge uma região sem defesa aérea (mísseis vermelhos, M2 e M4, na imagem ao final desta descrição)

1.2- O míssil atinge uma região com defesa aérea que já está esgotada (lembre-se: cada defesa tem uma capacidade limitada de defesa)

2. O míssil perde parte de seu poder de destruição e a cidade leva um dano igual ao poder restante do míssil. Isso ocorrerá nas seguintes situações:

   2.1- O míssil de ataque atinge uma região com defesa aérea com mísseis de defesa. A defesa gasta um míssil de defesa, porém com poder inferior ao poder original do míssil de ataque. O poder do míssil de defesa é descontado do poder do míssil de ataque (míssil amarelo, M3, na imagem ao final da descrição).

3. O míssil perde todo de seu poder de destruição e a cidade não leva nenhum dano. Isso ocorrerá nas seguintes situações:

   3.1- O míssil de ataque atinge uma região com defesa aérea com mísseis de defesa. A defesa gasta um míssil de defesa que possui poder maior ou igual ao míssil de ataque. Neste caso, o dano recebido pela cidade é zero (míssil cinza, M1, na imagem ao final da descrição).

- Após encerrar a entrada, deve gerar um relatório conforme descrição abaixo.

Restrições:

- Todas as variáveis que representam coordenadas e poder de ataque/defesa são do tipo **float.**
- Cada linha da entrada deve ser lida com apenas um scanf que termina com \n. **Olhe os exemplos!**
- Use alocação dinâmica para armazenar todas as variáveis dos tipos especificados pelos TADs e seus respectivos campos (exceto para os campos do tipo float especificados acima).
- O código deve seguir as interfaces definidas nos arquivos ".h" fornecidos junto com o exercício. Esses arquivos **não devem ser modificados**, pois contêm as especificações do problema a ser resolvido.
- Você será responsável pela manipulação correta da memória durante a execução do programa. Isso inclui a alocação dinâmica e a liberação de memória conforme necessário, garantindo que não haja vazamentos de memória.
- **Um mesmo míssil pode cair em uma região defendida por mais de uma defesa. Nesse caso, todas as defesas envolvidas gastam mísseis de defesa (se possuirem) e o míssil de ataque perde poder igual à soma do poder de todos os mísseis de defesa. Ex: Se um míssil com poder 100**

cai em uma região protegida por 2 defesas com poder 30, ele ficará com poder 40 e consequentemente causará dano 40 na cidade.

- **Não é necessário usar a biblioteca math.h nesse exercício. Caso você use, não esqueça do "-lm" na hora de compilar o código manualmente. O script já está preparado para usá-la.**

O relatório final deve conter as seguintes informações:

- Quantidade final de mísseis de defesa restantes de cada defesa aérea
- Dano causado por cada um dos mísseis lançados
- Porcentagem de efetividade de defesa, conforme fórmula abaixo.

Fórmulas e informações úteis

- Efetividade de defesa (%): $Efetividade = 100 \times \frac{dm - dr}{dm}$, onde dm seria a soma dos poderes de todos os mísseis que tem a cidade como alvo (ou o dano que a cidade receberia se não houvesse defesa) e dr é o dano total recebido pela cidade (após a defesa atuar).
- Verificar se a distância entre o centro da região (cx, cy) e o ponto do míssil (x,y) é menor ou igual ao raio (r) de atuação da defesa:

$$(x - c_x) \cdot (x - c_X) + (y - c_y)(y - c_y) \leq r \cdot r$$

- Você está recebendo alguns casos de teste. Há casos de teste ocultos que não serão executados no BOCA. Dessa forma, a nota que o script retornar no seu computador pode ser alterada devido a esse novo caso de teste.
- **Não esqueçam de incluir as bibliotecas que vocês usarem.**

Padrão de entrada

Seu programa deve ler da entrada padrão informações da cidade, defesas aéreas em mísseis, respeitando essa ordem. A cidade é representada pela primeira linha respeitando o padrão:

x y

Na sequência poderá existir uma ou mais defesas aéreas respeitando o seguinte padrão:

D
id x y tipo tamanho poder quantidade

Por fim, poderá existir um ou mais mísseis de ataque no seguinte padrão:

M

```
id x y poder
```

A entrada acaba com um caractere F. Um exemplo válido:

```
130.0 100.0
D
2 93.1 69.0 C 28.30 42.6 8
M
1 90.2 60.2 36.1
M
2 50.7 70.2 32.6
M
3 75 65 60.9
M
4 81 30 35.5
F
```

Nesta entrada, uma defesas foram cadastradas:

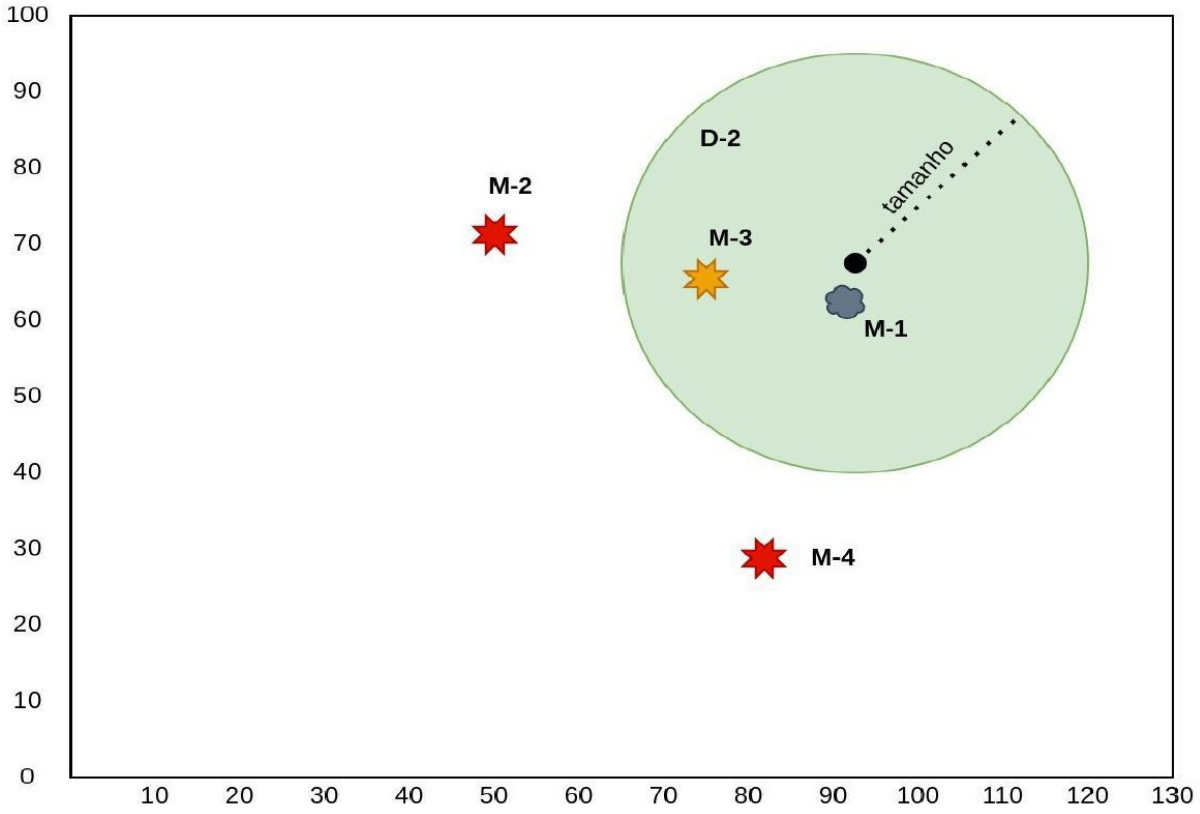
- D2 (circular), posicionada em (93.1, 69.0), com raio 28,3 e poder de defesa 42,6, com 7 mísseis de defesa disponíveis.

E quatro mísseis de ataque foram lançados:

- M1, com poder 36,1, posicionado em (90.2, 60.2)
- M2, com poder 32,6, posicionado em (50.7, 70.2)
- M3, com poder 60,9, posicionado em (75, 65)
- M4, com poder 35,5, posicionado em (81,30)

Como já dito, M2 e M4 caíram em regiões sem defesa, então causaram dano de 32,6 e 35,5, respectivamente. Já os outros:

- M1 tem poder 36,1 e caiu na região da defesa circular, com poder de defesa 42,6. Portanto, foi neutralizado e não causou nenhum dano à cidade.
- M3 tem poder 60,9 e caiu na região de defesa circular, que possui poder de defesa 42,6. Neste caso, o míssil causou dano de 18,3 (diferença dos poderes) à cidade.



A figura ilustra a área da cidade, a defesa aérea D-2 posicionadas, bem como sua zona de proteção. O míssil M-1 é totalmente neutralizado. M-2 e M-4 causam danos totais na cidade, e o M-3 é parcialmente neutralizado.

Assim, o relatório pode ser gerado conforme padrão abaixo:

<u>Padrão de saída</u>

A saída deverá ser impressa na tela (utilizando printf) de acordo com o padrão a seguir:

```
D-2: 6 D2 tinha 8 mísseis e gastou 2, então restaram 6
M-1: 0.00 M1 foi totalmente neutralizado
M-2: 32.60 M2 causou dano total
M-3: 18.30 M3 foi parcialmente neutralizado
M-4: 35.50 M4 causou dano total
Efetividade: 47.67% ( 100 * ((36,1+32,6+60,9+35,5) - (32,6 + 18,3 + 35,5)) / (165,1) )
                    100 * (soma_pod_misseis)     - ( soma_dano_causa)) / (so_p_mis)
```